

RETO 03 - RA2 - UT3

Visualizaciones Caja, Violín, Histograma, KDE y
Líneas

Ejercicio 1:

Gráficos de Caja (Box Plots)

Ejercicio 2:

Gráficos de Violín

Ejercicio 3:

Histogramas

Ejercicio 4:

KDE (Kernel Density Estimation)

Ejercicio 5:

Gráficos de Líneas

Antonio Ferrer Martínez

Marzo 2026

06b__Tarea__Graficos__de__Caja

March 14, 2026

1 Tarea paso a paso: Dominando los Gráficos de Caja (Box Plots)

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada paso a paso para que afiances los conceptos vistos en clase sobre la generación de gráficos de caja (box plots) mediante **Pandas y Seaborn** (basado en lo aprendido en `7_grafico_de_caja.ipynb`).

Nota importante sobre la Inteligencia Artificial: Es tentador copiar y pegar el enunciado de estos ejercicios en ChatGPT o Claude para que te den la solución de este Notebook en segundos. **Hacer eso perjudicará gravemente tu aprendizaje. (CODEA UN POCO QUE TE ENTERES QUE HACES)** En la vida real analítica (Data Science, Big Data), las IAs son asistentes geniales, pero el que diseña, entiende la arquitectura, detecta errores sutiles y toma las decisiones de valor eres *tú*. Si no entiendes qué hace un `boxplot`, qué parámetros recibe o cómo parametrizar los ejes X e Y, serás incapaz de resolver problemas de negocio reales y novedosos. **Usa tu cerebro, actúa paso a paso, equivócate, lee la documentación de Python, busca tu fallo y ¡aprende de él!**

1.1 Paso 1: Obtener el Dataset de Kaggle

Vamos a trabajar con un dataset muy popular e interesante para analizar distribuciones de notas escolares: **Student Performance in Exams**.

1. Regístrate o inicia sesión en Kaggle y entra a este enlace: [Student Performance in Exams](#)
2. Haz clic en *Download* (te descargará un archivo `.zip` que contiene un archivo CSV).
3. Extrae y renombra el archivo CSV a `students.csv` y muévelo directamente a tu subcarpeta `data/` (dentro de `02_Visualizaciones`).

1.2 Paso 2: Importar librerías

Importa `pandas`, `seaborn` y `matplotlib.pyplot` en la siguiente celda como hemos visto en clase.

```
[1]: # Importar la librería pandas para manipulación y análisis de datos
import pandas as pd
# Importar seaborn para visualización estadística atractiva
import seaborn as sns
# Importar pyplot de matplotlib para crear gráficos básicos y personalizar_
    ↪ figuras
import matplotlib.pyplot as plt
```

1.3 Paso 3: Carga de Datos y Exploración General

1. Usa `pandas` para leer el archivo CSV (`students.csv`) que has guardado en la carpeta `data/`.
2. Guárdalo en la variable de DataFrame típica (`df`) y muestra las primeras 5 filas (`.head()`) para ver la composición de sus columnas (`gender`, `race/ethnicity`, `math score`, etc).

```
[2]: # CARGA DE DATOS
# pd.read_csv() lee el archivo CSV (valores separados por comas) y lo convierte
# en un DataFrame (tabla de datos)
df = pd.read_csv("data/students.csv") # Por defecto detecta las columnas por el
# encabezado del archivo
# df.head() muestra las primeras 5 filas del DataFrame para tener una vista
# rápida de los datos
df.head()
```

```
[2]:  gender race/ethnicity parental level of education      lunch \
0  female      group B      bachelor's degree      standard
1  female      group C      some college      standard
2  female      group B      master's degree      standard
3   male      group A      associate's degree  free/reduced
4   male      group C      some college      standard

test preparation course  math score  reading score  writing score
0              none          72          72          74
1        completed          69          90          88
2              none          90          95          93
3              none          47          57          44
4              none          76          78          75
```

1.4 Paso 4: Gráfico de Caja con Pandas (`.boxplot`)

Objetivo: Analizar la distribución de las notas de matemáticas (`math score`) agrupados por el nivel educativo de los padres (`parental level of education`).

1. Llama a la función `.boxplot()` directamente sobre tu DataFrame `df`.
2. Utiliza el argumento `column` para indicar que queremos analizar numéricamente la columna `math score`.
3. Utiliza el argumento `by` para agrupar las cajas según `parental level of education`.
4. Gira las etiquetas del eje X si se superponen (puedes usar el parámetro `rot=45` o `rot=90` dentro de `boxplot`).
5. Observa el resultado. ¿Qué grupo tiene la mediana más alta y qué grupo tiene los valores atípicos (outliers) más bajos? Escribe un comentario en tu código.

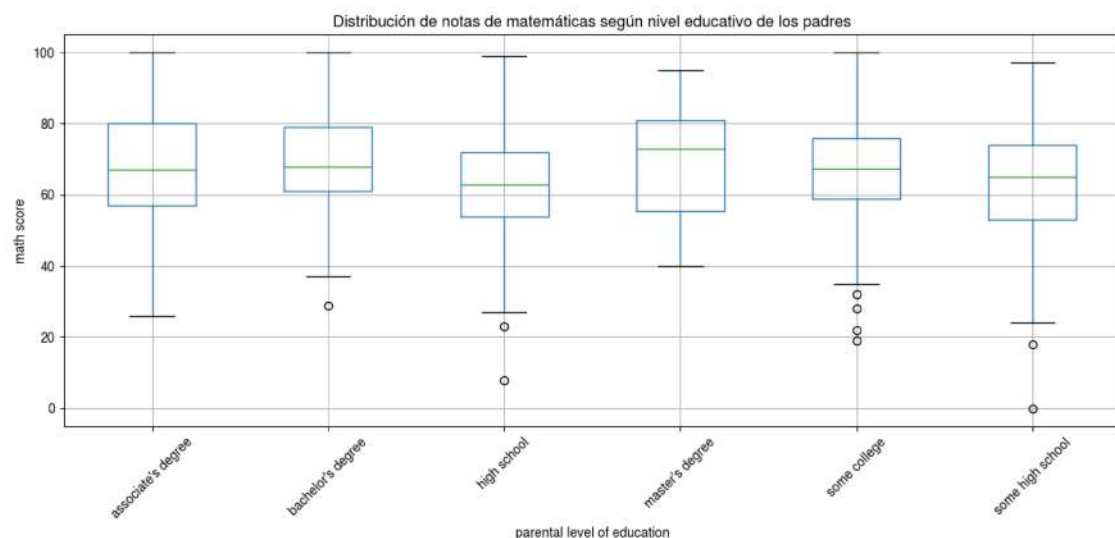
```
[3]: # Grafico de Cajas con Pandas (Matplotlib)
# https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.boxplot.html
# https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.boxplot.
# html#matplotlib.pyplot.boxplot
df.boxplot(
```

```

column='math score', # Columna numérica que queremos analizar
by='parental level of education', # Agrupamos las cajas según el nivel
↳ educativo de los padres
figsize=(12, 6), # Tamaño de la figura para que quepan bien las etiquetas
rot=45 # Rotamos las etiquetas del eje X para que no se superpongan
)
plt.title('Distribución de notas de matemáticas según nivel educativo de los
↳ padres')
plt.suptitle('') # Quitamos el título automático que pone Pandas
plt.ylabel('math score')
plt.tight_layout()
plt.show()

# El grupo "master's degree" tiene la mediana más alta
# Los valores atípicos más bajos aparecen en "some high school"

```



1.5 Paso 5: Gráfico de Caja con Seaborn (sns.boxplot)

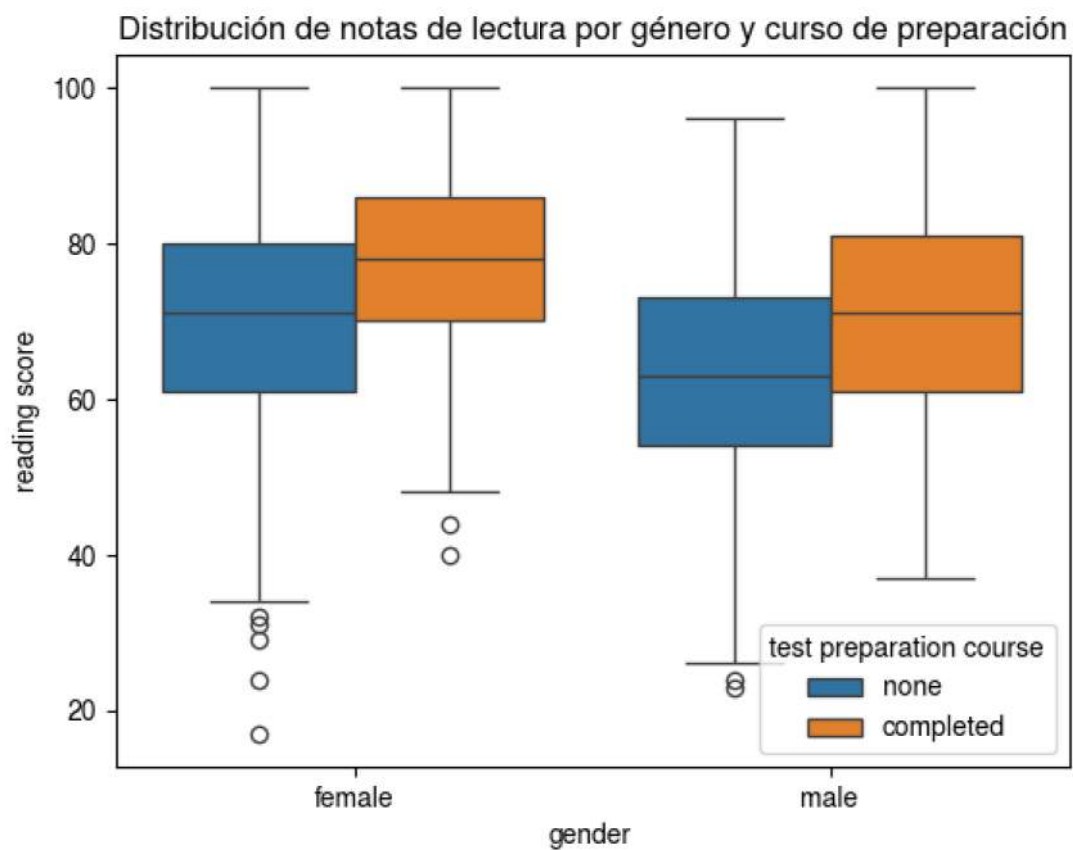
Objetivo: Analizar la distribución de las notas de lectura (**reading score**) según el género (**gender**), pero dividiendo además cada caja por si completaron o no el curso de preparación (**test preparation course**).

1. Llama a la función `sns.boxplot()`.
2. Pásale tu DataFrame `df` al parámetro `data`.
3. El eje X debe ser el género (`x='gender'`).
4. El eje Y debe ser la nota de lectura (`y='reading score'`).
5. Añade una dimensión categórica extra usando el argumento `hue='test preparation course'`.
6. Añade un título descriptivo a tu gráfico usando `.set(title='...')` al final de tu lla-

manejamiento a Seaborn.

```
[4]: # Grafico de Cajas con Seaborn
# https://seaborn.pydata.org/generated/seaborn.boxplot.html
sns.boxplot(
    data=df, # Pasamos el DataFrame completo
    x='gender', # Eje X: variable categórica (género)
    y='reading score', # Eje Y: variable numérica (nota de lectura)
    hue='test preparation course' # Dimensión categórica extra: si completaron
    o no el curso de preparación
).set(title='Distribución de notas de lectura por género y curso de
preparación')
```

```
[4]: [Text(0.5, 1.0, 'Distribución de notas de lectura por género y curso de
preparación')]
```



1.6 Reflexión Final del Alumno

Tómate un rato. Como has podido notar en esta lección:

- El uso de **diagramas de caja** es sumamente útil para detectar la mediana, la dispersión de

nuestros datos numéricos e identificar valores atípicos (outliers).

- Con **Pandas**, resulta muy fácil realizar un agrupamiento básico mediante el parámetro `by`.
- Con **Seaborn**, el parámetro `hue` nos empodera a añadir una dimensión categórica extra a nuestras distribuciones de manera muy limpia y estética.

Asegúrate que tu código de las celdas anteriores esté exhaustivamente comentado por ti en cada paso importante para justificar qué hace y demostrar que entiendes cómo las librerías generan las cajas, bigotes y puntos atípicos.

1.6.1 Mi reflexión

El boxplot con Pandas usando `by` es la forma más rápida de ver cómo se reparten las notas por grupo. Se ve enseguida dónde está la mediana y si hay valores que se salen mucho. Con Seaborn y `hue` se puede meter otra variable encima sin complicar demasiado el gráfico, en este caso se ve claro que preparar el examen influye bastante en la nota de lectura. Lo que más me ha servido es fijarme en los bigotes y los puntos sueltos para detectar outliers.

07b__Tarea__Graficos__de__Violin

March 14, 2026

1 Tarea paso a paso: Revelando Distribuciones con Gráficos de Violín

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada paso a paso para que afiances los conceptos vistos en clase sobre la generación de gráficos de violín (violin plots) mediante **Seaborn** (basado en lo aprendido en `7_grafico_de_violin.ipynb`).

Nota importante sobre la Inteligencia Artificial: Es tentador copiar y pegar el enunciado de estos ejercicios en ChatGPT o Claude para que te den la solución de este Notebook en segundos. **Hacer eso perjudicará gravemente tu aprendizaje. (CODEA UN POCO QUE TE ENTERES QUE HACES)** En la vida real analítica (Data Science, Big Data), las IAs son asistentes geniales, pero el que diseña, entiende la arquitectura, detecta errores sutiles y toma las decisiones de valor eres *tú*. Si no entiendes cómo interpretar la “barriga” de un violín o la utilidad brutal del parámetro `split`, no sabrás extraer el jugo a esta visualización. **Usa tu cerebro, actúa paso a paso, equivócate, lee la documentación de Python, busca tu fallo y ¡aprende de él!**

1.1 Paso 1: Obtener el Dataset de Kaggle

Seguiremos trabajando con el dataset de notas escolares: **Student Performance in Exams** que ya usaste para los gráficos de caja.

1. Si no lo tienes, descárgalo desde Kaggle: [Student Performance in Exams](#)
2. Recuerda extraer y renombrar el archivo CSV a `students.csv` y tenerlo directamente en tu subcarpeta `data/`.

1.2 Paso 2: Importar librerías

Importa `pandas`, `seaborn` y `matplotlib.pyplot` en la siguiente celda.

```
[1]: # Importar la librería pandas para manipulación y análisis de datos
import pandas as pd
# Importar seaborn para visualización estadística atractiva
import seaborn as sns
# Importar pyplot de matplotlib para crear gráficos básicos y personalizar_
↪ figuras
import matplotlib.pyplot as plt
```

1.3 Paso 3: Carga de Datos y Exploración General

1. Usa `pandas` para leer el archivo CSV (`students.csv`) desde la carpeta `data/`.
2. Guárdalo en la variable de DataFrame típica (`df`) y muestra las primeras 5 filas (`.head()`).

```
[2]: # CARGA DE DATOS
# pd.read_csv() lee el archivo CSV (valores separados por comas) y lo convierte
# en un DataFrame (tabla de datos)
df = pd.read_csv("data/students.csv") # Por defecto detecta las columnas por el
# encabezado del archivo
# df.head() muestra las primeras 5 filas del DataFrame para tener una vista
# rápida de los datos
df.head()
```

```
[2]:   gender race/ethnicity parental level of education      lunch \
0  female      group B      bachelor's degree    standard
1  female      group C      some college    standard
2  female      group B      master's degree    standard
3   male      group A  associate's degree  free/reduced
4   male      group C      some college    standard

   test preparation course  math score  reading score  writing score
0              none         72           72           74
1        completed         69           90           88
2              none         90           95           93
3              none         47           57           44
4              none         76           78           75
```

1.4 Paso 4: Gráfico de Violín Simple con Seaborn (`sns.violinplot`)

Objetivo: Analizar la forma de la distribución de las notas de matemáticas (`math score`) agrupadas por el nivel educativo de los padres (`parental level of education`). Queremos ver dónde se concentran la mayoría de las notas (el “grosor” del violín).

1. Llama a la función `sns.violinplot()`.
2. Pásale tu DataFrame `df`.
3. El eje X debe ser la categoría `parental level of education`.
4. El eje Y debe ser la variable numérica `math score`.
5. Si ves que los nombres del nivel educativo en el eje X se superponen y son ilegibles, usa la capa de `matplotlib` para girarlos: `plt.xticks(rotation=45, ha='right')`.
6. Observa el gráfico. Escribe un comentario en tu código sobre qué grupo educativo parece tener una distribución de notas de matemáticas más dispersa y alargada, y cuál está más “gordita” y concentrada en el centro.

```
[3]: # Violin Plot con Seaborn
# https://seaborn.pydata.org/generated/seaborn.violinplot.html
sns.violinplot(
    data=df, # Pasamos el DataFrame completo
```

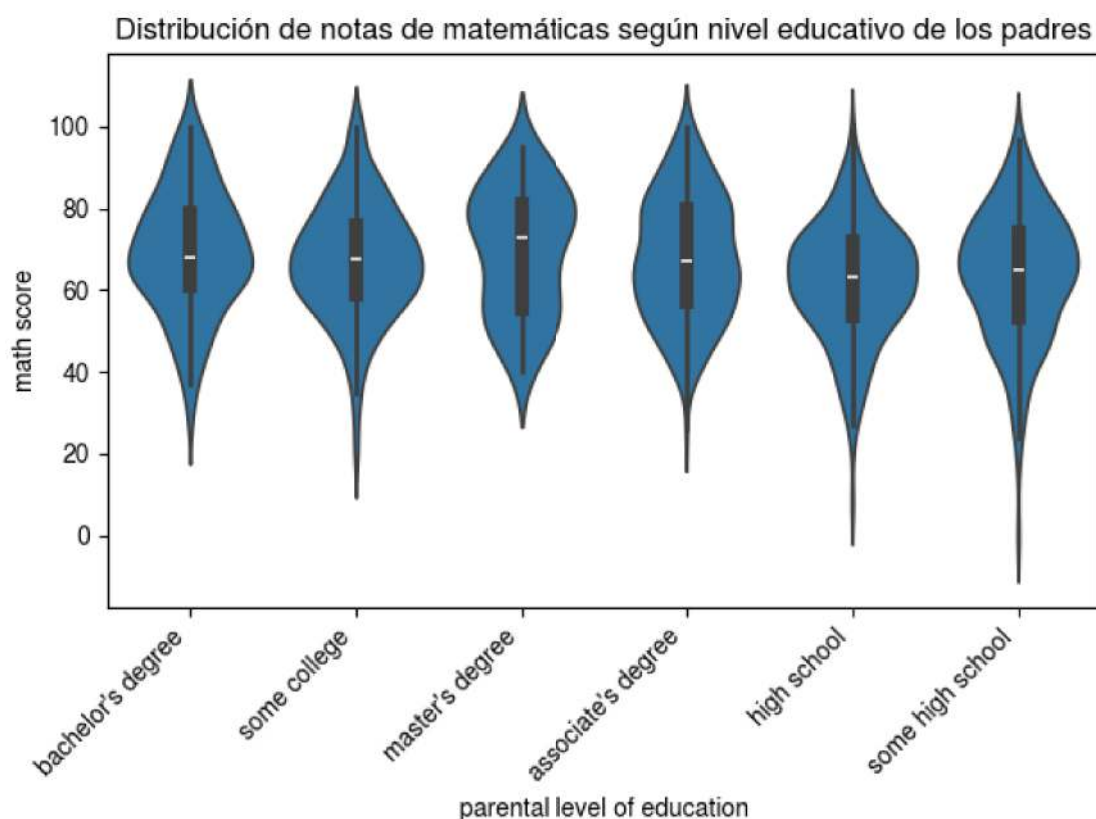


```

x='parental level of education', # Eje X: nivel educativo de los padres
y='math score', # Eje Y: variable numérica (nota de matemáticas)
).set(title='Distribución de notas de matemáticas según nivel educativo de los
↳padres')
# Giramos las etiquetas del eje X porque se superponen
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# El grupo "some high school" tiene una distribución más dispersa y alargada
# El grupo "master's degree" está más concentrado en el centro (violín más
↳gordito arriba)

```



1.5 Paso 5: El Superpoder del Violín: Argumento Hue y Split

Objetivo: Analizar la distribución de las notas de lectura (**reading score**) según el grupo étnico (**race/ethnicity**), separando internamente por género (**gender**) en el mismo cuerpo del violín.

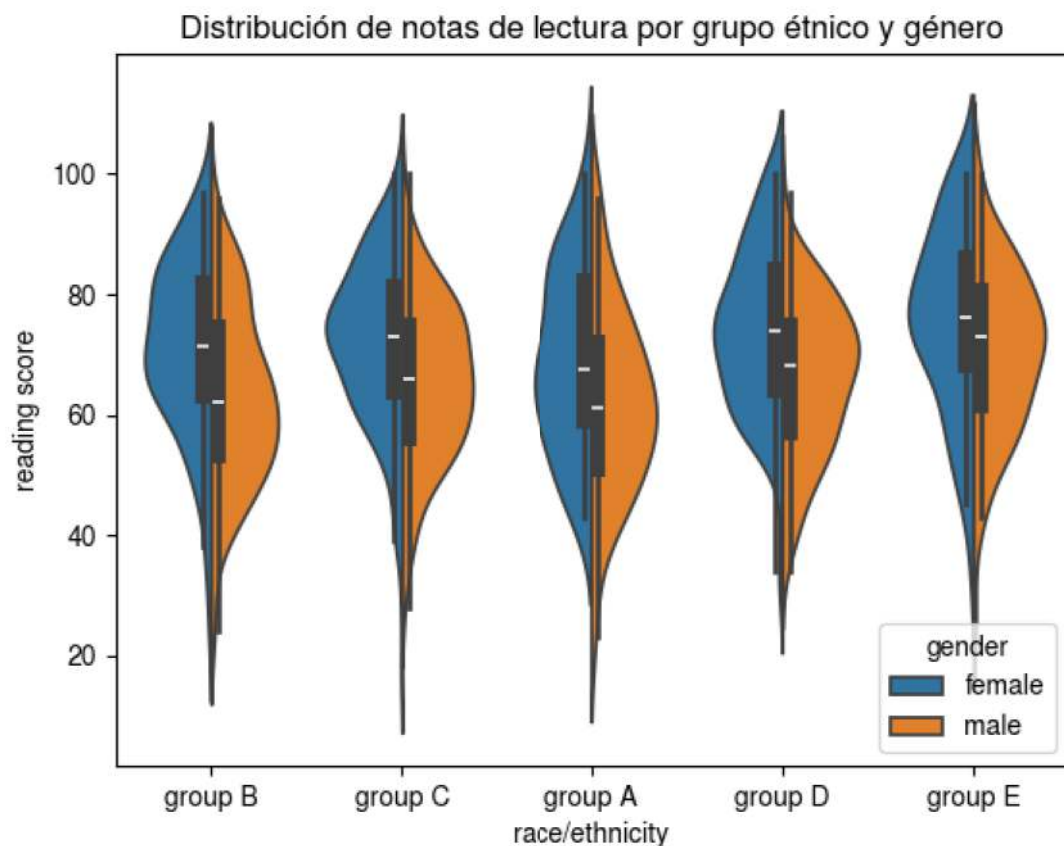
1. Llama de nuevo a la función `sns.violinplot()`.
2. El eje X será el grupo étnico (`x='race/ethnicity'`).
3. El eje Y será la nota de lectura (`y='reading score'`).

4. Añade la dimensión categórica del género usando el argumento `hue='gender'`. (Nota que género solo tiene 2 valores: 'male' y 'female').
5. **El truco mágico:** Añade el argumento `split=True`.
6. Añade un título descriptivo a tu gráfico.
7. Disfruta de la visualización obtenida. Ahora cada mitad del violín representa un género distinto dentro de cada etnia. Escribe un par de líneas comentadas sobre qué género tiende a “engordar” (concentrar más notas) en las franjas más altas dentro del ‘group E’ o ‘group C’.

```
[4]: # Violin Plot con Seaborn - usando hue y split
# https://seaborn.pydata.org/generated/seaborn.violinplot.html
sns.violinplot(
    data=df, # Pasamos el DataFrame completo
    x='race/ethnicity', # Eje X: grupo étnico
    y='reading score', # Eje Y: variable numérica (nota de lectura)
    hue='gender', # Dimensión categórica extra: género (male/female)
    split=True # Cada mitad del violín representa un género distinto
).set(title='Distribución de notas de lectura por grupo étnico y género')

# En el group E y group C las mujeres (female) tienden a concentrar más notas
# en las franjas altas, se ve porque su mitad del violín engor más arriba
```

```
[4]: [Text(0.5, 1.0, 'Distribución de notas de lectura por grupo étnico y género')]
```



1.6 Reflexión Final del Alumno

Tómate un rato. Como has podido notar en esta lección:

- El uso de **diagramas de violín** es una evolución estética y muy potente de las cajas, mostrándonos dónde exactamente residen las **masas de datos** (la Kde o *Kernel Density Estimation*).
- Si tuviéramos notas en forma de “reloj de arena” (muchos sacan 10, muchos sacan 0 y pocos sacan 5 - una distribución bimodal), un diagrama de caja pasaría esto por alto, pero el diagrama de violín nos lo chivaría al instante al tener dos “tripas” separadas.
- El parámetro `split=True` de Seaborn es exclusivo de los violines y es una de las herramientas de comparación de dos categorías enfrentadas más brutales que hay a nivel visual.

Asegúrate que tu código de las celdas anteriores esté exhaustivamente comentado por ti en cada paso importante para justificar qué hace y demostrar que lo entiendes.

1.6.1 Mi reflexión

El violín aporta más información que la caja porque muestra dónde se concentran realmente los datos. En el primer gráfico se ve que las notas de matemáticas no se reparten igual según el nivel de estudios de los padres. Con `split=True` y `hue='gender'` se pueden comparar dos grupos enfrentados en el mismo violín, que es mucho más visual que poner dos gráficos separados. La forma del violín ayuda a ver distribuciones bimodales que un boxplot no detectaría.

08b__Tarea__Histograma

March 14, 2026

1 Tarea paso a paso: Entendiendo Frecuencias con Histogramas

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada para que practiques y afiances cómo visualizar la **distribución de variables numéricas continuas** mediante Histogramas, utilizando tanto las funciones nativas de Pandas como Seaborn (basado en lo aprendido en `8_Histograma.ipynb`).

Nota importante sobre la Inteligencia Artificial: Como siempre, resiste la tentación de darle este enunciado a una IA para que te escupa la solución. **Si no te peleas con el código (escribiéndolo tú) y no juegas con los parámetros como el número de “bins”, no entenderás de verdad lo que un histograma te está intentando decir.** Las IAs te pueden ayudar a encontrar un error de sintaxis si te atascas, pero intenta razonar primero qué gráfico necesitas y qué variable va en cada eje. ¡A por ello!

1.1 Paso 1: Obtener el Dataset de Kaggle

Para esta tarea, vamos a usar un dataset muy famoso y divertido que contiene variables numéricas perfectas para medir distribuciones: **El dataset de Stats de Pokémon.**

1. Descárgalo desde Kaggle: [Pokemon with stats](#)
2. Extrae el archivo descargado. Pesa muy poco.
3. Renómbralo a `pokemon.csv` (para mantener la consistencia) y colócalo en tu carpeta `data/`.

1.2 Paso 2: Importar librerías

Importa `pandas`, `seaborn` y `matplotlib.pyplot`.

```
[1]: # Importar la librería pandas para manipulación y análisis de datos
import pandas as pd
# Importar seaborn para visualización estadística atractiva
import seaborn as sns
# Importar pyplot de matplotlib para crear gráficos básicos y personalizar
    ↪ figuras
import matplotlib.pyplot as plt
```

1.3 Paso 3: Carga de Datos y Exploración Inicial

1. Usa `pandas` para leer tu nuevo archivo `pokemon.csv`.
2. Guárdalo en el DataFrame `df` y muestra las primeras 5 filas con `.head()` para entender con qué columnas cuentas (fíjate especialmente en *HP*, *Attack*, *Defense*, *Speed*, *Legendary*).

```
[2]: # CARGA DE DATOS
# pd.read_csv() lee el archivo CSV (valores separados por comas) y lo convierte
# en un DataFrame (tabla de datos)
df = pd.read_csv("data/pokemon.csv") # Por defecto detecta las columnas por el
# encabezado del archivo
# df.head() muestra las primeras 5 filas del DataFrame para tener una vista
# rápida de los datos
df.head()
```

```
[2]: #
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	\
0	1	Bulbasaur	Grass	Poison	318	45	49	49	
1	2	Ivysaur	Grass	Poison	405	60	62	63	
2	3	Venusaur	Grass	Poison	525	80	82	83	
3	3	VenusaurMega	Venusaur	Grass	Poison	625	80	100	123
4	4	Charmander	Fire	NaN	309	39	52	43	


```

    Sp. Atk  Sp. Def  Speed  Generation  Legendary
0         65      65     45           1      False
1         80      80     60           1      False
2        100     100     80           1      False
3        122     120     80           1      False
4         60      50     65           1      False

```

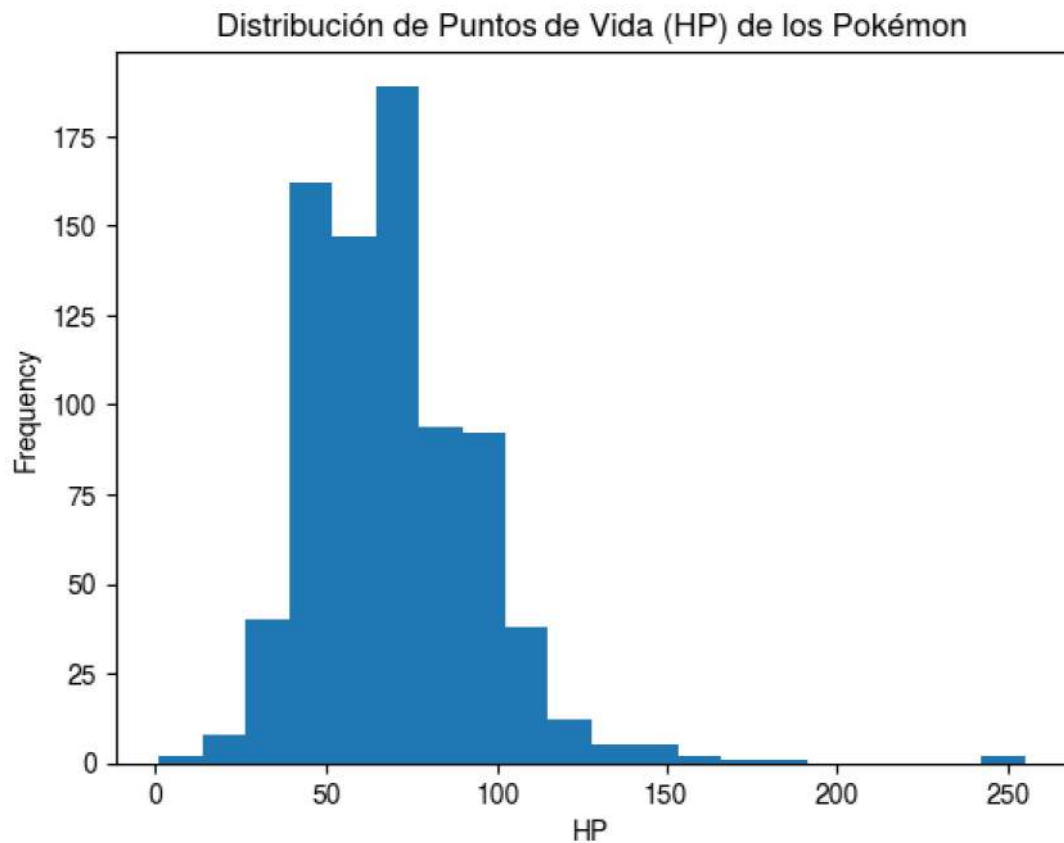
1.4 Paso 4: Tu Primer Histograma Nativo con Pandas

Objetivo: Queremos ver cómo se distribuyen los puntos de vida (HP) de todos los Pokémon. ¿Tienen la mayoría la misma vitalidad o hay mucha variedad?

1. Selecciona solo la columna HP de tu DataFrame.
2. Usa el método rápido de Pandas: `.plot.hist()`.
3. Por defecto, Pandas puede hacer las barras muy anchas (pocos “bins”). Pásale el argumento `bins=20` para dividir todo el rango de vida en 20 “cubos” o columnas.
4. Escribe un titular para el gráfico usando matplotlib.
5. Observa el gráfico generado. Deja un comentario: ¿Dónde dirías que se concentra la masa principal de Puntos de Vida de los Pokémon?

```
[3]: # Histograma con Pandas (Matplotlib)
# https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.hist.html
# https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.hist.
# html#matplotlib.pyplot.hist
df['HP'].plot.hist(
    bins=20 # Dividimos el rango de HP en 20 contenedores
)
plt.title('Distribución de Puntos de Vida (HP) de los Pokémon')
plt.xlabel('HP')
plt.show()
```

```
# La masa principal de Puntos de Vida se concentra entre 40 y 80 HP
↪ aproximadamente
```



1.5 Paso 5: Histograma con Seaborn y Curva de Densidad (KDE)

Los métodos de Pandas son rápidos, pero Seaborn (`sns.histplot()`) ofrece opciones estadísticas geniales al vuelo.

Objetivo: Analizar la distribución del Ataque (`Attack`) y dibujar suavemente su forma (curva de densidad).

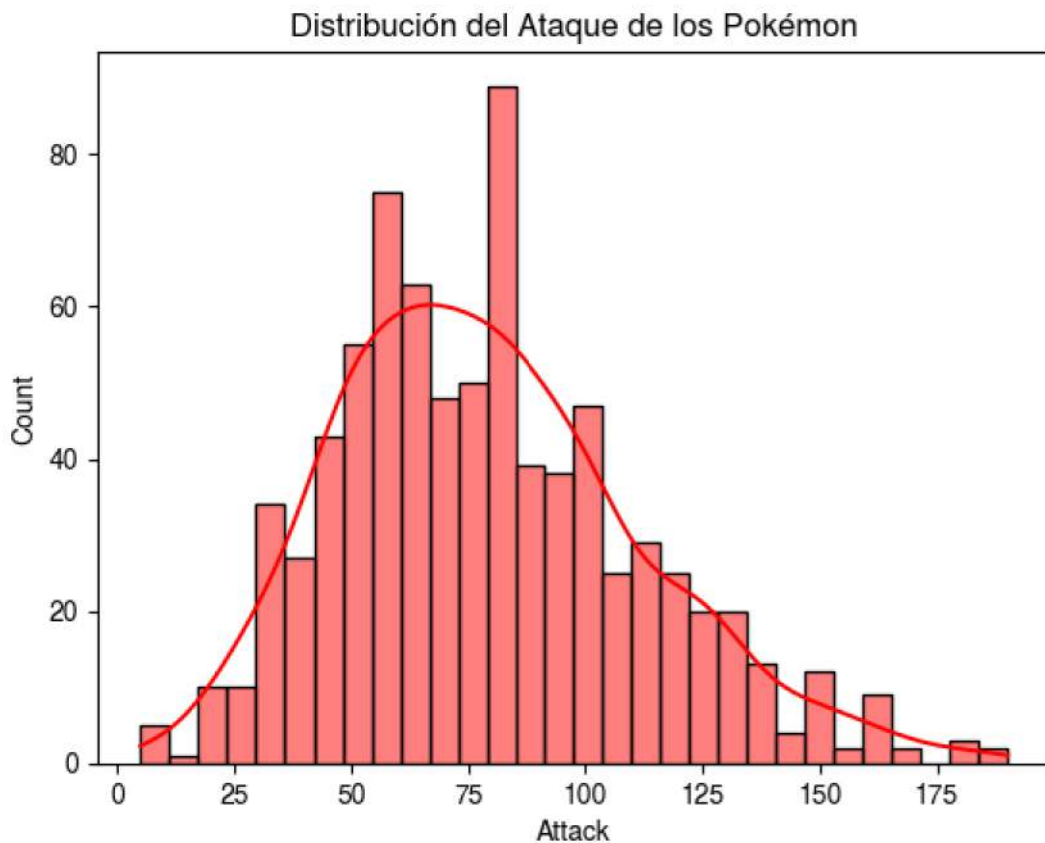
1. Llama a `sns.histplot()`.
2. Pásale tu DataFrame (`df`).
3. El eje de análisis numérico será el Ataque (`x='Attack'`).
4. Añade el parámetro clave para generar la campana de distribución de densidad: `kde=True`.
5. Si quieres, colorea las barras añadiendo `color='red'` o el que prefieras, y ajusta los `bins=30`.
6. Añade título descriptivo.

```
[4]: # Histograma con Seaborn y curva de densidad (KDE)
# https://seaborn.pydata.org/generated/seaborn.histplot.html
sns.histplot(
```

```

data=df, # Pasamos el DataFrame completo
x='Attack', # Eje X: variable numérica (Ataque)
kde=True, # Añadimos la curva de densidad suavizada
bins=30, # Número de contenedores
color='red' # Color de las barras
)
plt.title('Distribución del Ataque de los Pokémon')
plt.show()

```



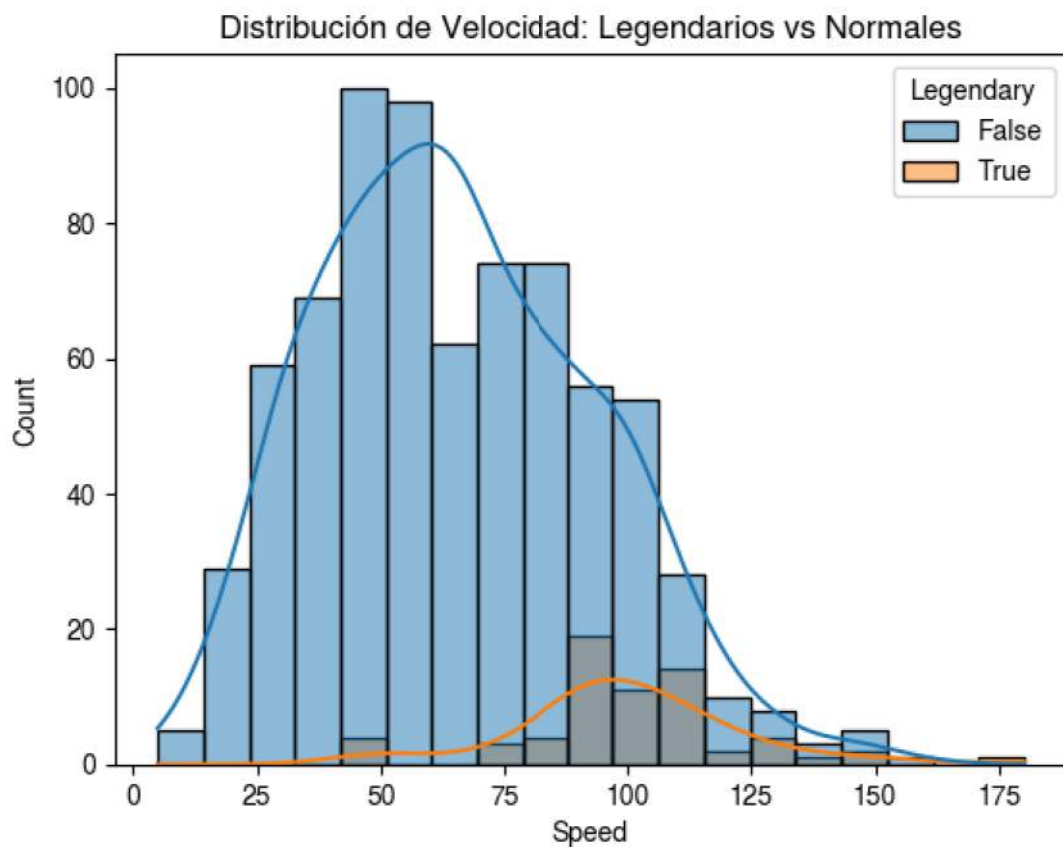
1.6 Paso 6: Comparación de distribuciones (El súper útil Hue)

Objetivo: Queremos comprobar un mito: ¿Son los Pokémon Legendarios (Legendary) estadísticamente más rápidos (Speed) que los Pokémon normales?

1. Llama a `sns.histplot()` con `x='Speed'`.
2. Usa el argumento `hue='Legendary'` para que Seaborn separe automáticamente los datos en dos histogramas superpuestos (verdadero y falso).
3. Añade la curva de densidad si quieres (`kde=True`).
4. Fíjate en el gráfico resultante e interprétalo. Escribe un comentario en tu código diciendo si hay una clara tendencia a que los Legendarios tengan barras amontonadas más hacia la derecha (valores altos de velocidad).

```
[5]: # Histograma comparativo con Seaborn usando hue
# https://seaborn.pydata.org/generated/seaborn.histplot.html
sns.histplot(
    data=df, # Pasamos el DataFrame completo
    x='Speed', # Eje X: variable numérica (Velocidad)
    hue='Legendary', # Separamos automáticamente en dos histogramas
    # ↳ superpuestos (True/False)
    kde=True # Añadimos la curva de densidad
)
plt.title('Distribución de Velocidad: Legendarios vs Normales')
plt.show()

# Los Pokémon Legendarios tienen sus barras más desplazadas a la derecha
# lo que indica que tienden a ser más rápidos que los normales
```



1.7 Reflexión Final del Alumno

Comprende lo que has hecho:

- A diferencia del gráfico de barras que cuenta “cuántos hay en una categoría”, el **histograma** agarra una **variable numérica continua** (como el salario de una persona, la temperatura

media de las ciudades, o en este caso los Stats de un Pokémon), trocea todo el rango posible matemáticamente desde el mínimo hasta el máximo en “contenedores” (**bins**) y cuenta la frecuencia que cae en caja cajita.

- Jugar con la cantidad de **bins** es vital: muy pocos ocultan cómo es la forma real, y demasiados generan “ruido” y barras vacías tipo peine roto.
- La curva KDE nos ayuda a idealizar esa forma para ver rápidamente si sigue una distribución normal (campana) clásica.

¡Asegúrate de haber comentado las deducciones o respuestas de los gráficos en las propias celdas de código!

1.7.1 Mi reflexión

El histograma divide el rango numérico en bins y cuenta cuántos valores caen en cada uno. La cantidad de bins cambia mucho la lectura: con 20 se ve bien la forma general de HP, con 30 el Attack ya tiene más detalle. La curva KDE encima del histograma ayuda a ver si la distribución se parece a una campana o no. Lo más útil ha sido usar `hue='Legendary'` para comparar Pokémon legendarios y normales en el mismo gráfico, se nota que los legendarios tienen un Speed bastante más alto.

09b__Tarea__KDE

March 14, 2026

1 Tarea paso a paso: Modelando Densidades con KDE

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada para que domines otra de las grandes herramientas de la Data Science visual: los **Gráficos de Densidad Kernel (KDE)**. Estos gráficos te enseñarán a mirar más allá de los ruidosos “bloques” de los histogramas para apreciar la verdadera matemática detrás de los datos.

Ojo con la IA: Sabemos que ya lo sabes, pero te lo recordamos. Si copias y pegas las preguntas en herramientas automáticas, perderás por completo el *feeling* intuitivo de lo que hace el ancho de banda y perderás destreza manual con pandas/seaborn. Escribe tú el código. Si falla, léelo con calma y usa la IA sólo como un corrector ortográfico de código si te bloqueas. ¡Tu mente analítica te lo agradecerá en el examen!

1.1 Paso 1: El Famoso Dataset de los Pingüinos de Palmer

Adiós a la tripulación del Titanic. Hola a los reyes del hielo. Para entender las distribuciones continuas, vamos a analizar pingüinos en una isla de la Antártida.

1. Descarga el dataset de Kaggle: [Palmer Archipelago \(Antarctica\) penguin data](#).
2. Si te descarga varios archivos, localiza el CSV simplificado que suele llamarse `penguins_size.csv` o similar.
3. Renómbralo a `penguins.csv` para comodidad, y archívalo obligatoriamente en tu carpeta `data/`.

1.2 Paso 2: Importar librerías

Como siempre, prepara tus herramientas: `pandas`, `seaborn` y `matplotlib.pyplot`.

```
[1]: # Importar la librería pandas para manipulación y análisis de datos
import pandas as pd
# Importar seaborn para visualización estadística atractiva
import seaborn as sns
# Importar pyplot de matplotlib para crear gráficos básicos y personalizar
↪ figuras
import matplotlib.pyplot as plt
```

1.3 Paso 3: Carga y Limpieza Temprana (¡Cuidado con los Nulos!)

1. Usa `pd.read_csv()` para cargar `penguins.csv` en tu variable de siempre, `df`.
2. Muestra las primeras filas con `.head()`.

3. **Paso Clave:** En la realidad, las mediciones a veces fallan. Es muy posible que en este dataset algunas filas tengan valores NaN (nulos). Puesto que vamos a graficar curvas matemáticas continuas, lo mejor ahora mismo es limpiar la basura.

- Escribe en tu celda de abajo `df = df.dropna()` para eliminar permanentemente las filas defectuosas y que no nos fastidien los cálculos de densidad.

```
[2]: # CARGA DE DATOS
# pd.read_csv() lee el archivo CSV (valores separados por comas) y lo convierte
# en un DataFrame (tabla de datos)
df = pd.read_csv("data/penguins.csv") # Por defecto detecta las columnas por el
# encabezado del archivo
# Eliminamos las filas con valores nulos para que no fastidien los cálculos de
# densidad
df = df.dropna()
# df.head() muestra las primeras 5 filas del DataFrame para tener una vista
# rápida de los datos
df.head()
```

```
[2]: species      island  culmen_length_mm  culmen_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen         39.1             18.7             181.0
1  Adelie  Torgersen         39.5             17.4             186.0
2  Adelie  Torgersen         40.3             18.0             195.0
4  Adelie  Torgersen         36.7             19.3             193.0
5  Adelie  Torgersen         39.3             20.6             190.0

   body_mass_g  sex
0      3750.0  MALE
1      3800.0 FEMALE
2      3250.0 FEMALE
4      3450.0 FEMALE
5      3650.0  MALE
```

1.4 Paso 4: Tu Primer Gráfico KDE con Seaborn

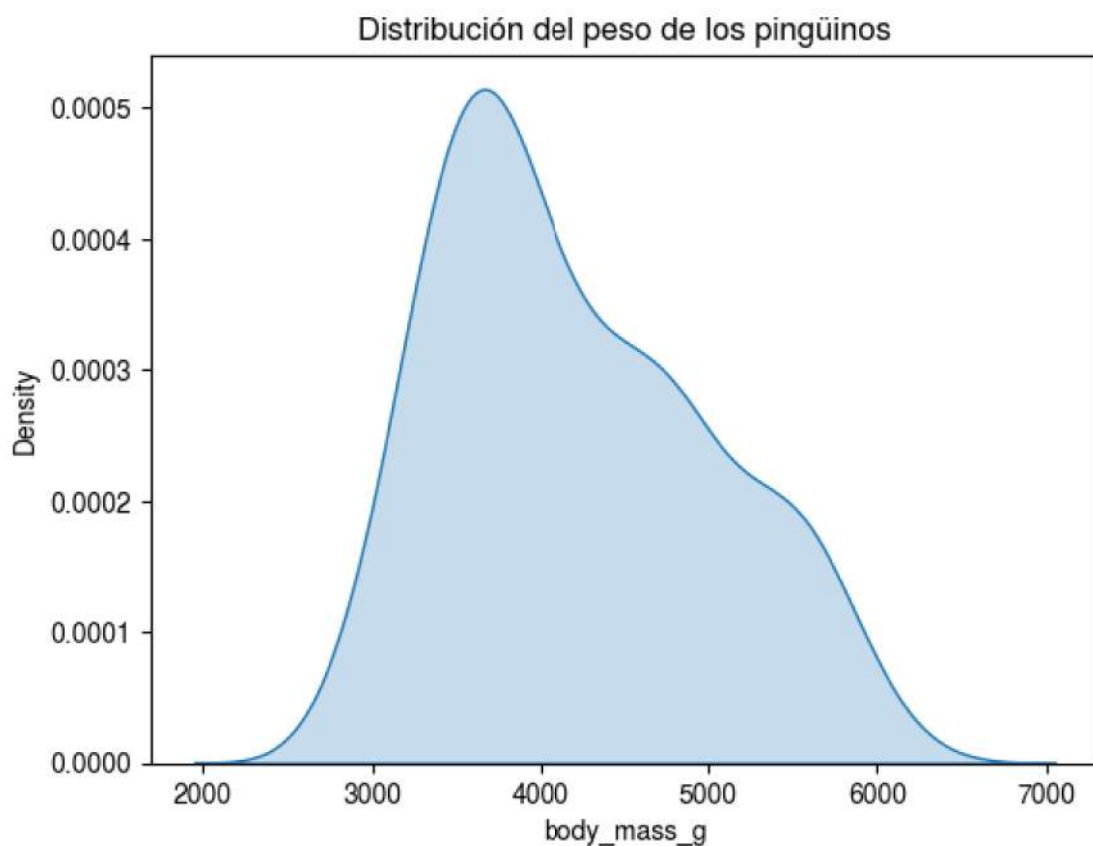
Objetivo: Visualizar de forma continua y elegante cómo se distribuye el peso de los pingüinos de esta isla.

1. Usa Seaborn llamando a la función `sns.kdeplot()`.
2. Asigna la fuente de datos `data=df`.
3. Escoge la columna del peso corporal en el eje de las X (`x='body_mass_g'`).
4. Para que no sea sólo una triste línea, añade el parámetro visual `fill=True` para rellenar la curva de color.
5. Observa y comenta: ¿Aprecias una “subida” gorda al principio y otra más sutil más adelante? Esto indica que es una distribución bimodal. Hay dos “grupos” principales de pesos distintos mezclados.

```
[3]: # KDE con Seaborn
# https://seaborn.pydata.org/generated/seaborn.kdeplot.html
```

```
sns.kdeplot(
    data=df, # Pasamos el DataFrame completo
    x='body_mass_g', # Eje X: peso corporal en gramos
    fill=True # Rellenamos la curva de color para que no sea solo una línea
)
plt.title('Distribución del peso de los pingüinos')
plt.show()

# Se aprecia una subida gorda al principio (alrededor de 3500g) y otra más
↳ sutil más adelante (alrededor de 5000g)
# Esto indica que es una distribución bimodal: hay dos grupos principales de
↳ pesos distintos mezclados
```



1.5 Paso 5: Domando la Suavidad (El parámetro `bw_adjust`)

El KDE intenta adivinar la curva, pero tú puedes decirle si quieres que sea muy “nerviosa/receptiva” a los datos o muy “relajada/suavizada”. Esto se hace con el ancho de banda (`bw_adjust`). Por defecto vale 1.

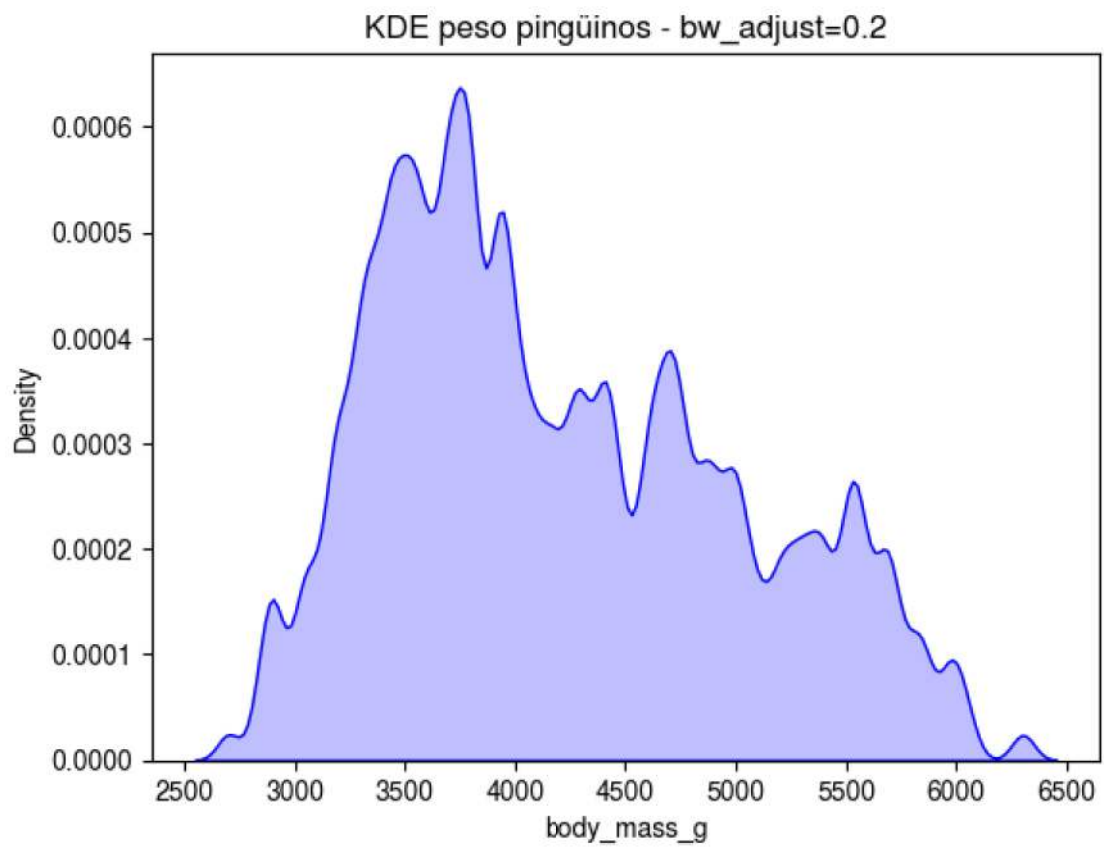
Objetivo: Ver cómo el ancho de banda destruye o desvela el detalle.

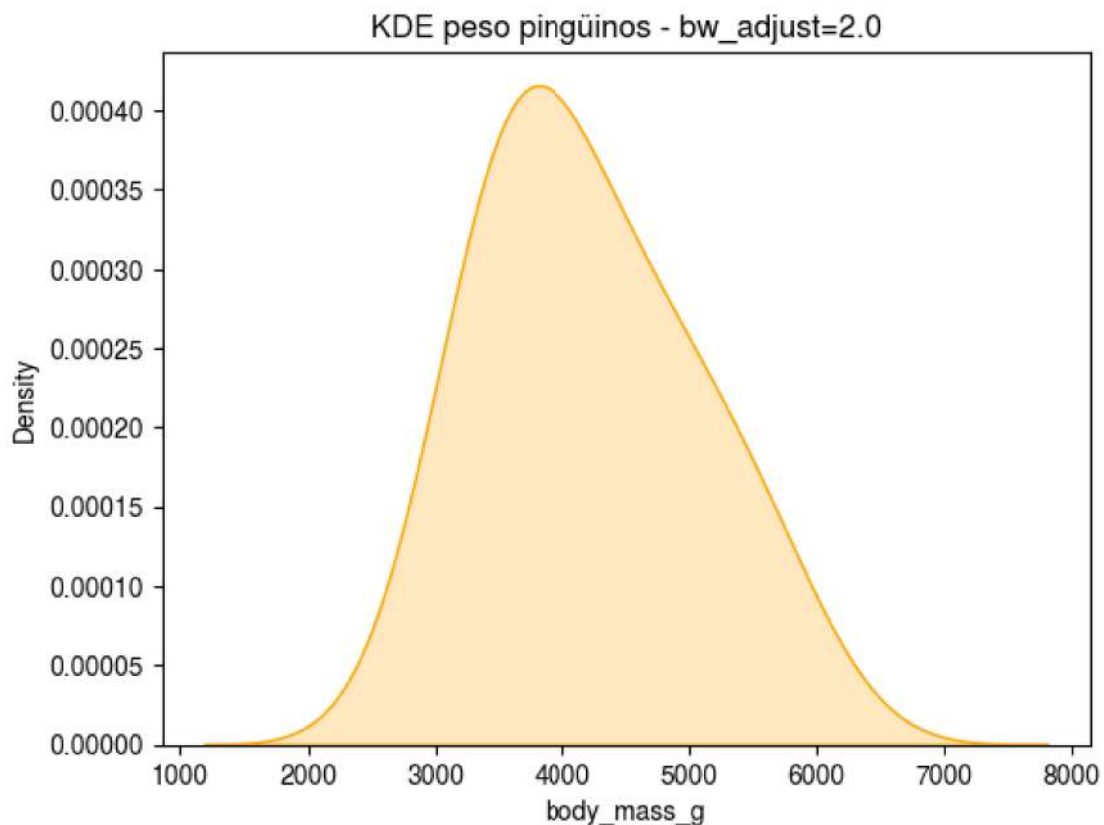
1. Vuelve a escribir el código exacto del Paso 4 (puedes copiar el código anterior).
2. Pero ahora añádele al KDE el parámetro `bw_adjust=0.2`.
3. Debajo en la misma celda, repite el código pero con `bw_adjust=2.0`.
4. (Opcional) Si quieres diferenciarlos, dales distintos colores con `color='blue'` o similar.
5. Comenta en la celda: ¿Qué efecto tiene un valor de `bw_adjust` cercano a 0? ¿Sigues pareciendo una distribución útil y elegante o parece ruido?

```
[4]: # KDE con bw_adjust bajo (0.2) - muy receptivo a los datos, mucho detalle
# https://seaborn.pydata.org/generated/seaborn.kdeplot.html
sns.kdeplot(
    data=df,
    x='body_mass_g',
    fill=True,
    bw_adjust=0.2, # Ancho de banda bajo: curva muy nerviosa/receptiva
    color='blue'
)
plt.title('KDE peso pingüinos - bw_adjust=0.2')
plt.show()

# KDE con bw_adjust alto (2.0) - muy suavizado, pierde detalles
sns.kdeplot(
    data=df,
    x='body_mass_g',
    fill=True,
    bw_adjust=2.0, # Ancho de banda alto: curva muy relajada/suavizada
    color='orange'
)
plt.title('KDE peso pingüinos - bw_adjust=2.0')
plt.show()

# Con bw_adjust=0.2 la curva parece ruido, demasiados picos y no se ve la forma
# ↪ real
# Con bw_adjust=2.0 se suaviza tanto que pierde el detalle de la distribución
# ↪ bimodal
```





1.6 Paso 6: El Poder Definitivo del KDE (Comparar Especies con Hue)

Objetivo: En lugar de ver todos los pingüinos amasados, queremos separar la distribución del largo de las aletas (`flipper_length_mm`) en función de qué Especie son (`species`).

1. Llama a `sns.kdeplot()` con `x='flipper_length_mm'`.
2. El truco maestro: usa `hue='species'` para generar una curva matemática por cada especie de la isla.
3. No olvides el `fill=True`.
4. El solapamiento puede ser un problema escondiendo datos. Solúcnalo dando transparencia al relleno con el parámetro `alpha=0.5`.
5. Deja un comentario de análisis de datos real: ¿Qué especie tiene la curva más separada a la derecha demostrando que tienen las aletas más enormes?

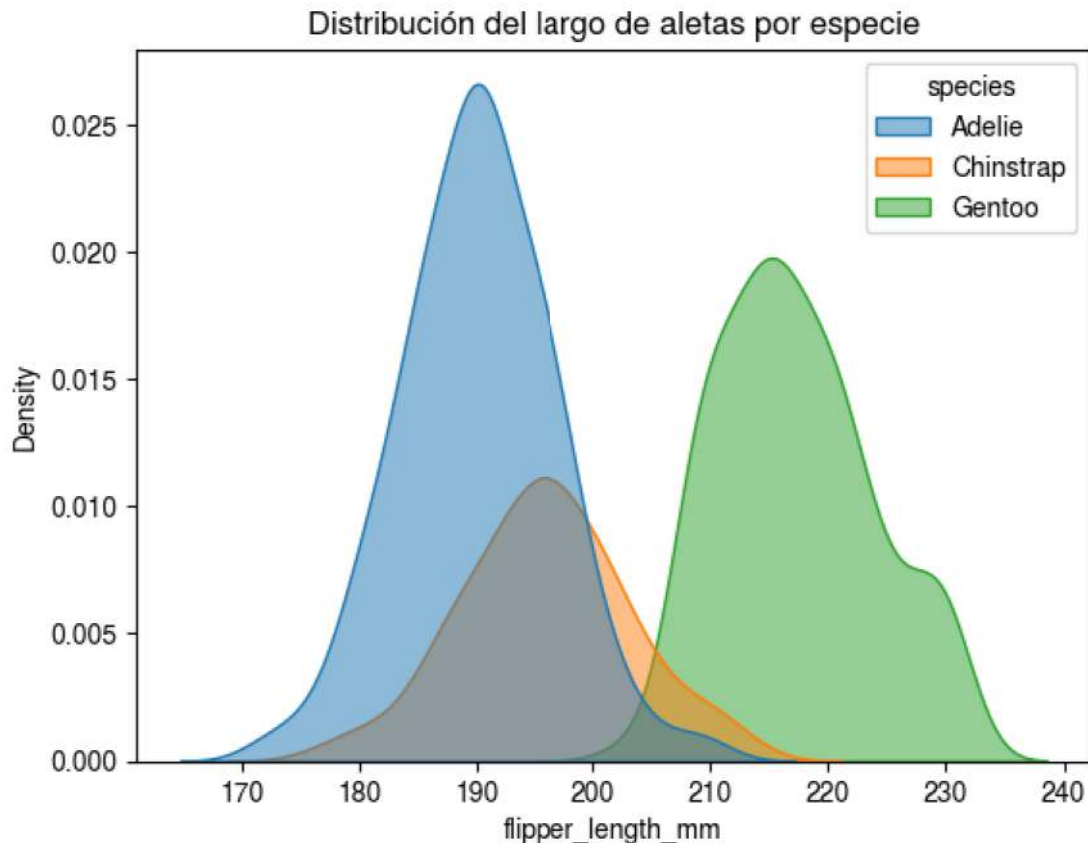
```
[5]: # KDE superpuesto por especies con Seaborn
# https://seaborn.pydata.org/generated/seaborn.kdeplot.html
sns.kdeplot(
    data=df, # Pasamos el DataFrame completo
    x='flipper_length_mm', # Eje X: largo de las aletas en mm
    hue='species', # Genera una curva por cada especie
    fill=True, # Rellenamos las curvas de color
```

```

    alpha=0.5 # Transparencia al 50% para que no se tapen entre sí
)
plt.title('Distribución del largo de aletas por especie')
plt.show()

# La especie Gentoo tiene la curva más separada a la derecha
# lo que demuestra que tienen las aletas más grandes

```



1.7 Reflexión Final del Alumno

Terminando el Bloque:

- Piensa en un KDE como tirar una “sábana elástica” por encima de los polvorientos y escalonados bloques de un Histograma para ver la forma suave y general del terreno.
- Gracias a esa “sábana” (curva de densidad), somos capaces de solapar múltiples KDEs en un solo gráfico (como has visto en el último paso). Si hubiéramos intentado pintar varios Histogramas de barras encimados, hubiera sido un desastre visual opaco e ininteligible. El KDE nos libra de las barreras al analizar la superposición de datos continuos.
- Acuérdate del “bw_adjust”: no asumas ciegamente que la curva por defecto de Python es la 100% correcta. De ti depende como analista ajustarla si crees que está esquivando el ruido demasiado y perdiéndose detalles sutiles.

1.7.1 Mi reflexión

El KDE es como una versión suavizada del histograma, sin los bloques escalonados. Con `bw_adjust` se controla cuánto suaviza: a 0.2 se ve mucho detalle (demasiado ruido) y a 2.0 se pierde información pero queda la tendencia general. Lo más interesante ha sido superponer las tres especies de pingüinos con `hue='species'`, se distingue bien que cada especie tiene un rango diferente de `flipper_length`. El `dropna()` al principio es importante porque si hay nulos el KDE no funciona bien.

10b_Tarea_Grafico_de_Lineas

March 14, 2026

1 Tarea paso a paso: Viajando en el Tiempo con Gráficos de Líneas

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada para que sumes a tu cinturón una habilidad esencial en el mundo del Dato: **El manejo de las series temporales (Time Series)** a través de los Gráficos de Líneas simples y multilíneas.

Ojo con la IA: Entender cómo procesar el tiempo en Pandas y cómo leer las tendencias no se puede subcontratar de forma pasiva. Teclea el código tú mismo/a. Analiza los picos y valles antes de avanzar. Si algo falla, intenta interpretarlo o busca el error metódicamente. ¡Este es el formato rey en los cuadros de mando de analítica de negocio!

1.1 Paso 1: El Histórico de Bolsa FAANG (Facebook, Apple, Amazon, Netflix, Google)

Para hablar del tiempo y tendencias vamos a ir a lo directo: la economía.

1. Descarga el dataset de Kaggle: [FAANG Historical Stock Price Data](#).
2. Descarga los datos y si vienen separados, localiza idealmente un único CSV que las reúna, renómbralo a `faang_stocks.csv` y archívalo obligatoriamente en tu carpeta `data/`. *(Si hubiera un CSV por cada empresa en Kaggle, júntalos o escoge el de Apple -AAPL- para el paso 4 y el de Google -GOOGL- e intenta la fusión en Pandas luego, aunque lo ideal para esta clase es buscar uno que venga unificado en Kaggle).*

1.2 Paso 2: Importar librerías

Nuestras aliadas inseparables: `pandas`, `seaborn` y `matplotlib.pyplot`.

```
[1]: # Importar la librería pandas para manipulación y análisis de datos
import pandas as pd
# Importar seaborn para visualización estadística atractiva
import seaborn as sns
# Importar pyplot de matplotlib para crear gráficos básicos y personalizar
↳ figuras
import matplotlib.pyplot as plt
```

1.3 Paso 3: Carga y Conversión del Tiempo (Time Series Magic)

1. Usa `pd.read_csv()` para cargar `faang_stocks.csv` en tu variable `df`.

2. Muestra las primeras filas para familiarizarte con columnas como la fecha (Date), el Ticker (símbolo de la empresa) y el precio al cerrar (Close).
3. **Paso Clave para el Tiempo:** Los CSV son texto; Pandas no sabe mágicamente que “2020-01-01” es 1 de Enero y no un código extraño.
 - Transforma la columna escribiendo `df['Date'] = pd.to_datetime(df['Date'])`. Esto hará que en tus gráficos de líneas Seaborn sea capaz de situar los años a la perfección en el Eje X.

```
[2]: # CARGA DE DATOS
# El dataset de Kaggle venía con un CSV por cada empresa (Apple.csv, Amazon.
#     ↪ csv, etc.)
# Como dice el enunciado, los junto en uno solo añadiendo una columna Ticker

archivos = {
    'AAPL': 'data/Apple.csv',
    'AMZN': 'data/Amazon.csv',
    'META': 'data/Facebook.csv',
    'GOOG': 'data/Google.csv',
    'NFLX': 'data/Netflix.csv'
}

lista = []
for ticker, ruta in archivos.items():
    temp = pd.read_csv(ruta)
    temp['Ticker'] = ticker # Añadimos columna con el símbolo de la empresa
    lista.append(temp)

# pd.concat() une varios DataFrames en uno solo, uno debajo de otro
df = pd.concat(lista, ignore_index=True)
# Guardamos el resultado como faang_stocks.csv en la carpeta data/
df.to_csv('data/faang_stocks.csv', index=False)

# Transformamos la columna Date al tipo datetime para que Pandas la reconozca
#     ↪ como fecha
# Los CSV son texto; Pandas no sabe mágicamente que "2020-01-01" es 1 de Enero
df['Date'] = pd.to_datetime(df['Date'])
# df.head() muestra las primeras 5 filas del DataFrame para tener una vista
#     ↪ rápida de los datos
df.head()
```

```
[2]:
```

	Date	Open	High	Low	Close	Adj Close	Volume	\
0	1980-12-12	0.128348	0.128906	0.128348	0.128348	0.101261	469033600.0	
1	1980-12-15	0.122210	0.122210	0.121652	0.121652	0.095978	175884800.0	
2	1980-12-16	0.113281	0.113281	0.112723	0.112723	0.088934	105728000.0	
3	1980-12-17	0.115513	0.116071	0.115513	0.115513	0.091135	86441600.0	
4	1980-12-18	0.118862	0.119420	0.118862	0.118862	0.093777	73449600.0	

```

    Ticker
0    AAPL
1    AAPL
2    AAPL
3    AAPL
4    AAPL

```

1.4 Paso 4: El Ascenso de una Acción (Gráfico de Línea Simple)

Objetivo: Ver la evolución histórica en solitario del éxito estelar (o no) de una única empresa tecnológica.

1. Primero, aísla un poco los datos. Crea una variable nueva llamada `df_apple` filtrando tu DataFrame original para quedarte solo con los registros donde el `Ticker` (o la columna que la identifique) sea `'AAPL'`.
2. Llama a la función `sns.lineplot()`.
3. Informa de los datos (`data=df_apple`), pon tu fantástica columna temporal en el eje inferior (`x='Date'`) y pide que te dibuje la variación del precio de cierre en la vertical (`y='Close'`).
4. Añade un comentario en la celda analizando visualmente: ¿Se aprecian “valles” claros (caídas de precio) en algún año en concreto o la progresión de esta empresa parece un cohete imparable?

```

[3]: # Filtramos el DataFrame para quedarnos solo con Apple (AAPL)
df_apple = df[df['Ticker'] == 'AAPL']

# Grafico de Lineas con Seaborn
# https://seaborn.pydata.org/generated/seaborn.lineplot.html
sns.lineplot(
    data=df_apple, # Pasamos el DataFrame filtrado solo con Apple
    x='Date', # Eje X: columna temporal
    y='Close' # Eje Y: precio de cierre
)
plt.title('Evolución histórica del precio de Apple (AAPL)')
plt.show()

# Se aprecian algunos valles claros (caídas de precio) pero en general
# la tendencia de Apple es un crecimiento bastante constante a largo plazo

```



1.5 Paso 5: El Gráfico Multilínea (Batalla del FAANG)

El poder real analítico viene de la comparación, y aquí el gráfico de barras se rinde. Vamos a poner a competir a las 5 gigantes.

Objetivo: Combinar varias líneas cronológicas usando la magia del parámetro de color categorizador.

1. Vuelve a emplear `sns.lineplot()` pero esta vez alimentándolo con tu dataframe completo de los 5 Titanes (`data=df`). Mantén `x='Date'` e `y='Close'`.
2. Añade el parámetro clave para separar las líneas automáticamente y generar una leyenda perfecta: `hue='Ticker'`.
3. Ajusta o cuida tu gráfico. (*Opcional*) Si las leyendas molestan, la función te ayuda. Pero lo vital es mirar el resultado.
4. Responde en un comentario final: Mirando la carrera entre el inicio y el final de la serie temporal, ¿qué Línea/Empresa ha crecido de la forma más descomunal según su color, destrozando a sus rivales en precio nominal?

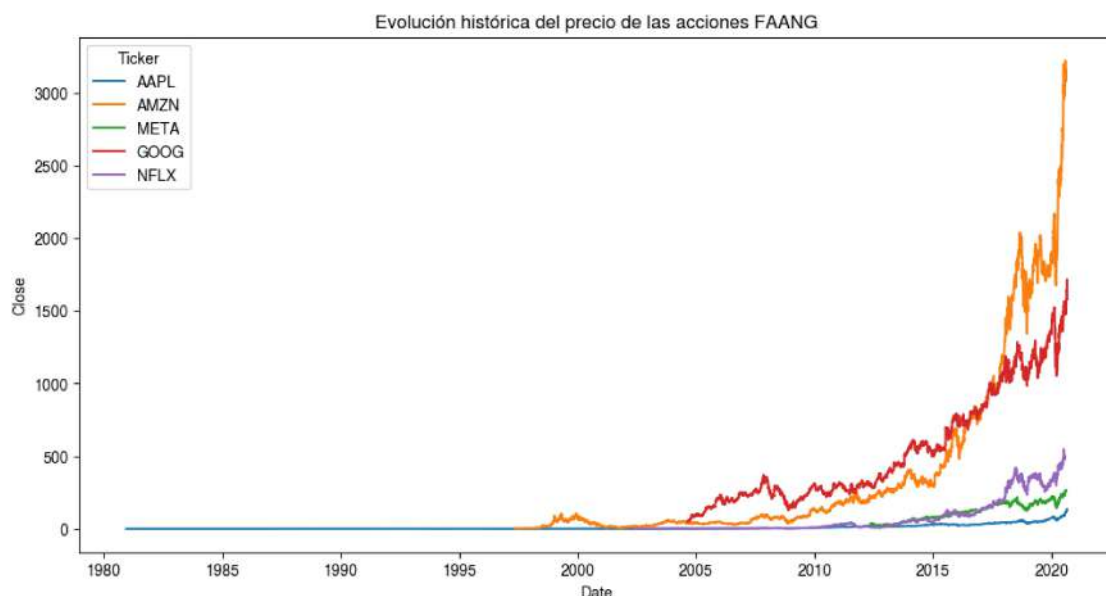
```
[4]: # Grafico de Lineas Multilínea con Seaborn
# https://seaborn.pydata.org/generated/seaborn.lineplot.html
plt.figure(figsize=(12, 6)) # Ampliamos la figura para que se vea bien
```

```

sns.lineplot(
    data=df, # Pasamos el DataFrame completo con las 5 empresas
    x='Date', # Eje X: columna temporal
    y='Close', # Eje Y: precio de cierre
    hue='Ticker' # Separamos las líneas por empresa, genera una leyenda
    ↪ automática
)
plt.title('Evolución histórica del precio de las acciones FAANG')
plt.show()

# La empresa que más ha crecido de forma descomunal es Amazon (AMZN)
# su línea destaca por encima de todas las demás en precio nominal

```



1.6 Reflexión Final del Alumno

Terminando el Bloque:

- Piensa en un Gráfico de Barras para sumar categorías estáticas (¿Cuántos perros vs gatos?).
- Piensa en un **Gráfico de Líneas** para seguir el hilo de una historia, rastreando la conexión lógica de picos o progresiones, que en el 99% de las ocasiones tienen sentido gracias a ese ingrediente silencioso en el horizonte del Eje X: *El paso inquebrantable del Tiempo*.
- La conversión de formato fecha (`pd.to_datetime`) es tu seguro de vida para no desquiciar los años de tus series analíticas.

1.6.1 Mi reflexión

El gráfico de líneas es el que mejor funciona cuando el eje X es temporal. Lo primero que hay que hacer es convertir la columna Date con `pd.to_datetime`, si no Pandas la trata como texto y el eje

queda desordenado. Con Apple sola ya se ve una tendencia clara de subida con algún valle puntual. Al juntar las 5 empresas con `hue='Ticker'` se compara fácil cuál ha crecido más, y Amazon destaca bastante por encima del resto en precio nominal. El paso de unir los CSV separados en uno solo fue necesario porque Kaggle los tenía por empresa.